

Agenda

- Comparator
- Collection Framework
- Traversal
- FailSafe and FailFast Iterator
- ~~List~~

Collection Framework

- Collection framework is Library of reusable data structure classes that is used to develop application.
- Main purpose of collection framework is to manage data/objects in RAM efficiently.
- Collection framework was introduced in Java 1.2 and type-safe implementation is provided in 5.0 (using generics).
- Collection is available in java.util package.
- Java collection framework provides

1. Interfaces -- defines standard methods for the collections.

2. Implementations -- classes that implements various data structures.

3. Algorithms -- helper methods like searching, sorting, ...

Collection Hierarchy

- Interfaces: Iterable, Collection, List, Queue, Set, Map, Deque, SortedSet, SortedMap, ...
- Implementations: ArrayList, LinkedList, HashSet, HashMap, ...
- Algorithms: sort(), reverse(), max(), min(), ... -> in Collections class static methods

Collection interface

- Root interface in collection framework interface hierarchy.
- Most of collection classes are inherited from this interface (indirectly).
- Provides most basic/general functionality for any collection
- Abstract methods
 - boolean add(E e)
 - int size()
 - boolean isEmpty()
 - void clear()
 - boolean contains(Object o)
 - boolean remove(Object o)
 - boolean addAll(Collection<? extends E> c)
 - boolean containsAll(Collection<?> c)
 - boolean removeAll(Collection<?> c)
 - boolean retainAll(Collection<?> c)
 - Object[] toArray()
 - Iterator iterator() -- inherited from Iterable
- Default methods
 - default Stream stream()

- default Stream parallelStream()
- default boolean removeIf(Predicate<? super E> filter)

Iterable interface

- To traverse any collection it provides an Iterator.
- Enable use of for-each loop.
- In java.lang package
- Iterable yeilds an iterator
- Methods
 - Iterator iterator()
 - default Splitter splitter()
 - default void forEach(Consumer<? super T> action)

Iterator (Demo01)

- Part of collection framework (1.2)
- Methods
 - boolean hasNext()
 - E next()
 - void remove()

Collections class- Since Java 1.0

- Helper/utility class that provides several static helper methods
- Methods
 - List reverse(List list);
 - List shuffle(List list);
 - void sort(List list, Comparator cmp)
 - E max(Collection list, Comparator cmp);
 - E min(Collection list, Comparator cmp);
 - List synchronizedList(List list);

Collection vs Collections

1. Collection interface

- All methods are public and abstract. They implemented in sub-classes.
- Since all methods are non-static, must be called on object.

```
Collection<Integer> list = new ArrayList<>();  
//List<Integer> list = new ArrayList<>();  
//ArrayList<Integer> list = new ArrayList<>();  
list.remove(new Integer(12));
```

2. Collections class

- Helper class that contains all static methods.

- We never create object of "Collections" class.

```
Collections.methodName(...);
```

Fail-fast vs Fail-safe Iterator

- If state of collection is modified (add/remove operation other than iterator methods) i.e structural change while traversing a collection using iterator and iterator methods fails (with `ConcurrentModificationException`), then iterator is said to be Fail-fast.
- The collections from `java.util` package have fail-fast iterators
- e.g. Iterators from `ArrayList`, `LinkedList`, `Vector`, ...
- If iterator allows to modify the underlying collection (add/remove operation other than iterator methods) while traversing a collection (NO `ConcurrentModificationException`), then iterator is said to be Fail-safe.
- The collections from `java.util.concurrent` package have fail-safe iterators.
- e.g. Iterators from `CopyOnWriteArrayList`, ...
- If any changes are done in the collection using these iterators then the changes may not be reflected using the same iterator however by creating the new iterator we can get the changes displayed.

`java.util` => fail fast iterator
`java.util.concurrent` => fail safe iterator

Traversal

1. Using Iterator

```
Iterator<Integer> itr = list.iterator();
while(itr.hasNext()) {
    Integer i = itr.next();
    System.out.println(i);
}
```

2. Using for-each loop

```
for(Integer i:list)
    System.out.println(i);

// gets converted into Iterator traversal internally

for(Iterator<Integer> itr = list.iterator(); itr.hasNext();) {
    Integer i = itr.next();
    System.out.println(i);
}
```

3. using for loop

```
for(int index=0; index<list.size(); index++) {  
    Integer i = list.get(index);  
    System.out.println(i);  
}
```

Assignment

- Create a student class with fields rollno,name,marks and course. provide natural ordering of elements on rollno. Create an array of 10 students and Write a menu driven code that has below menus.

1. Add Student
2. display all students sorted on rollno
3. display all students sorted on name
4. display all students sorted on marks in desc order
5. display all students sorted on course